

CS101: Operating Systems — Concurrency and Process Management

1. Introduction to Processes and Threads

In modern operating systems, a process is an instance of a program in execution. It contains the program code, its current activity represented by the program counter, stack, data section, and heap memory. In contrast, a thread is the smallest unit of execution within a process, often termed a lightweight process. Threads belonging to the same process share the process's address space, open file descriptors, and global variables, but each maintains its own private stack and register state.

2. Process Scheduling & Dispatching

The CPU scheduler selects among the processes in memory that are ready to execute. Common scheduling algorithms include First-Come-First-Served (FCFS), Shortest Job Next (SJN), Priority Scheduling, and Round Robin (RR). In Round Robin scheduling, each ready process is assigned a fixed time slice called a quantum (typically 10 to 100 milliseconds). If a process does not finish within its quantum, it is preempted and returned to the tail of the ready queue. Modern Linux kernels employ the Completely Fair Scheduler (CFS), which uses red-black trees to balance CPU time among tasks proportionally based on nice values.

3. Synchronization Primitives and Race Conditions

When multiple concurrent threads access shared data without synchronization, race conditions occur where the outcome depends on the non-deterministic order of execution. Critical sections are code segments accessing shared resources. Mutual exclusion ensures that only one thread executes in its critical section at any given time. Semaphores, introduced by Edsger Dijkstra, are integer variables accessed via two atomic operations: `wait()` (or P) and `signal()` (or V). A binary semaphore behaves like a mutex lock, whereas a counting semaphore allows a fixed number of threads to access a finite pool of resources.

4. Deadlocks and Coffman Conditions

A deadlock occurs when a set of concurrent processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process. According to Coffman et al. (1971), four conditions must hold simultaneously for a deadlock to arise: 1) Mutual Exclusion: resources cannot be shared; 2) Hold and Wait: processes hold at least one resource while waiting for others; 3) No Preemption: resources cannot be forcibly taken from a process holding them; 4) Circular Wait: a closed chain of processes exists where each process waits for a resource held by the next. Deadlock avoidance algorithms such as Dijkstra's Banker's Algorithm dynamically inspect resource allocation state to ensure safe execution states.